🎥 Google Meet Video Recording Script (Side-by-Side Presentation Guide)

DISTRIBUTED URBAN AIR QUALITY ANALYTICS & FORECASTING (DISTRIBUTED BIG DATA & ML) • NATURAL PRESENTATION WALKTHROUGH

Distributed Architecture • HDFS → Spark → Spark MLlib

Teammate A (Driver / Storage Lead)
Shares screen, runs commands, presents Steps 1 & 2 (HDFS Data Lake) & Step 7 Conclusion

Teammate B (Spark ETL & Analytics Lead)
Presents Steps 3 & 4 (Distributed Spark ETL, CPCB 16h validity & 3 Diurnal/AQI Charts)

Teammate C (Spark MLlib ML Lead)
Presents Steps 5 & 6 (Chronological 24h Forecasting +16.6% & 6-Tier Hazard Confusion Matrix)

VIDEO INTRODUCTION (30 SECONDS) • TEAMMATE A (SCREEN SHARER)

*"Hello everyone. In this video, our team is presenting our Big Data project: **Distributed Urban Air Quality Analytics and AQI Forecasting**. We implemented the full distributed lifecycle under a **Distributed Big Data Architecture**—spanning **Hadoop HDFS → Apache Spark distributed ETL → Spark MLlib**. We analyze continuous telemetry from Delhi NCR, Bengaluru, and Hyderabad adhering strictly to Indian **CPCB regulatory standards**. Let us begin by inspecting our raw distributed data lake in Hadoop HDFS."*

🎬 Step 1: Inspecting Raw Telemetry in Hadoop HDFS

Speaker: Teammate A (Driver / Storage Lead)

🕹️ TERMINAL ACTION (WHAT DRIVER RUNS)

```
./1_hdfs_show_raw.sh
```

🖥️ SCREEN OUTPUT (WHAT IS DISPLAYED)

```
$ hdfs dfs -ls -h /air_quality/raw/
Found 1 items
-rw-r--r--   1 yashsonawane supergroup     47.3 M 2026-10-14 23:13 /air_quality/raw/raw_hourly_measurements.jsonl
```

🗣️ WHAT TO SAY NATURALLY

"Here in our first step, our command just listed the raw telemetry staged inside Hadoop HDFS under the path `/air_quality/raw/`.

As shown on the terminal, we have **raw_hourly_measurements.jsonl**, which is **47.3 Megabytes** and contains **96,000 raw continuous hourly records** collected from the OpenAQ API across Delhi, Bengaluru, and Hyderabad.

Our client respects OpenAQ's 60 req/min rate limit with automated token replenishment. Staging this in HDFS allows Hadoop to distribute the data using standard 128 MB blocks and a 3x replication factor.

Now, let us inspect Step 2 to see how we store the processed data."

Key Highlights to Point at:

- Point to **47.3 M**: Real continuous telemetry (96k rows), not dummy sample data.
- Point to **/air_quality/raw/**: Demonstrates proper HDFS distributed file zoning.

🎬 Step 2: Inspecting Partitioned Parquet Lake in HDFS

Speaker: Teammate A (Driver / Storage Lead)

🕹️ TERMINAL ACTION (WHAT DRIVER RUNS)

```
./2_hdfs_show_parquet.sh
```

🖥️ SCREEN OUTPUT (WHAT IS DISPLAYED)

```
$ hdfs dfs -ls -h /air_quality/processed/hourly_aqi_parquet/
Found 4 items
-rw-r--r--   1 yashsonawane supergroup          0 2026-10-15 02:22 /air_quality/pro
cessed/hourly_aqi_parquet/_SUCCESS
drwxr-xr-x   - yashsonawane supergroup          0 2026-10-15 02:22 /air_quality/pro
cessed/hourly_aqi_parquet/city=Bengaluru
drwxr-xr-x   - yashsonawane supergroup          0 2026-10-15 02:22 /air_quality/pro
cessed/hourly_aqi_parquet/city=Delhi NCR
drwxr-xr-x   - yashsonawane supergroup          0 2026-10-15 02:22 /air_quality/pro
cessed/hourly_aqi_parquet/city=Hyderabad
```

🗣️ WHAT TO SAY NATURALLY

"Moving to Step 2, this is our processed data lake in HDFS.

Notice that instead of plain CSV files, we persist our cleaned data as **columnar Apache Parquet with Snappy compression**, partitioned by city—`city=Bengaluru`, `city=Delhi NCR`, and `city=Hyderabad`.

This gives us significant performance gains through **Partition Pruning**. When downstream Spark queries run on Delhi data, Spark scans only the Delhi directory and skips all other cities, saving disk I/O and network shuffle.

Now, my teammate will explain Step 3 and demonstrate our distributed Spark ETL."

Key Highlights to Point at:

- Point to **_SUCCESS**: Hadoop output committer validation mark.
- Point to **city=XXX folders**: Automatic Hive-style partitioning for fast parallel scans.

🎬 Step 3: Running Distributed Apache Spark ETL & CPCB Math

Speaker: Teammate B (Spark ETL & Analytics Lead)

🕹️ TERMINAL ACTION (WHAT DRIVER RUNS)

```
./3_spark_run_etl.sh
```

🖥️ SCREEN OUTPUT (WHAT IS DISPLAYED)

```
$ python3 scripts/spark_etl.py
================ STARTING DISTRIBUTED SPARK ETL ================
Apache Spark Version: 4.2.0
[1/6] Ingesting raw dataset from: hdfs://localhost:9000/air_quality/raw/...
  Ingested 96,000 raw records.
[3/6] Normalizing timestamps to IST and harmonizing pollutant units...
[4/6] Reshaping and pivoting pollutants into hourly station records...
[5/6] Calculating rolling exposure windows and CPCB NAQI...
  Total Station-Hour Records Processed: 18,521
  Records Meeting Strict CPCB 16h NAQI Validity: 12,792

--- Sample Processed Station Records with CPCB AQI ---
+---------+----------------------+------------------+------------------+--------+
---------------+
|city     |station_name          |ts_ist            |pm25_24h          |cpcb_aqi|
health_category|
+---------+----------------------+------------------+------------------+--------+
---------------+
|Delhi NCR|R K Puram, Delhi - DPCC|2016-02-06 12:30:00|193.42857142857142|500.0   |
Severe         |
+---------+----------------------+------------------+------------------+--------+
---------------+
Spark ETL Pipeline completed successfully in 20.71 seconds.
```

🗣️ WHAT TO SAY NATURALLY

(While running): "The driver has started our Apache Spark ETL script. Spark 4.2.0 is loading the 96,000 raw records from HDFS into an in-memory DataFrame."

(When output finishes): "The pipeline completed in just 20 seconds. Here is what took place:

1. In Step 3, we converted gases like NO2 and SO2 from parts-per-billion into µg/m³ using their molecular weights.
2. In Step 4, Spark pivoted individual pollutant streams into **18,521 station-hour records**.
3. In Step 5, we enforced the official Indian **CPCB 16-hour completeness rule**: a 24-hour moving average is only valid if at least 16 hourly readings exist. Exactly **12,792 records** met this strict rule.
4. In the sample table, you can see calculated 24h PM2.5, PM10, and final AQI—R K Puram shows an AQI of 500, which falls into the **Severe** category.

Next, let us run Step 4 to inspect our Spark SQL analytics and visual trends."

🎬 Step 4: Distributed Spark SQL Analytics & Summary Tables

Speaker: Teammate B (Spark ETL & Analytics Lead)

🕹️ TERMINAL ACTION (WHAT DRIVER RUNS)

```
./4_spark_run_analytics.sh
```

🖥️ SCREEN OUTPUT (WHAT IS DISPLAYED)

```
$ python3 scripts/run_analytics.py
================ RUNNING DISTRIBUTED ANALYTICS ================
[1/4] Calculating city-wise descriptive statistics...
     city  total_hours  valid_aqi_hours  avg_pm25  std_pm25  avg_pm10  avg_cpcb_aqi
max_cpcb_aqi
Delhi NCR         8131             5924     121.9      97.7     288.8         382.0
500.0
Bengaluru         6126             3378      28.8      52.7      68.9         293.9
500.0
Hyderabad         4264             3490      28.0      21.6      69.7         265.1
500.0

[2/4] Computing CPCB health category distribution by city...
     city health_category  category_count pct_of_city
Delhi NCR          Severe            3701        62.5
Bengaluru    Satisfactory            1236        36.6
Hyderabad    Satisfactory            1440        41.3

[4/4] Rendering analytics visualizations...
  [COMMAND 4 - FIGURE 1] Saved: reports/figures/cmd4_fig1_diurnal_pm25_trends.png
  [COMMAND 4 - FIGURE 2] Saved: reports/figures/cmd4_fig2_cpcb_category_distributio
n.png
  [COMMAND 4 - FIGURE 3] Saved: reports/figures/cmd4_fig3_city_aqi_boxplots.png
```

🗣️ WHAT TO SAY NATURALLY

"Our Spark SQL distributed aggregations are now displayed on screen:

* **Table 1:** Looking at the city averages, Delhi's average PM2.5 is **121.9 µg/m³**, whereas Bengaluru is only **28.8** and Hyderabad is **28.0**. Delhi's particulate pollution is over four times higher.

* **Table 2:** In Delhi, a massive **62.5% of all monitored hours fell into the Severe category**. In contrast, Bengaluru has 36.6% Satisfactory hours and Hyderabad has 41.3%.

* **Generated Figures:** Below the tables, Spark generated three figures for Command 4. Let us inspect each generated chart side-by-side with its findings."

Key Highlights to Point at:

- Point to **Delhi avg_pm25 (121.9)** vs **Bengaluru (28.8)**.
- Point to **Delhi Severe % (62.5%)** proving severe northern winter smog.

Step 4 - Figure 1: Diurnal Pattern of PM2.5 Concentration across Metros

Visual Reference: cmd4_fig1_diurnal_pm25_trends.png

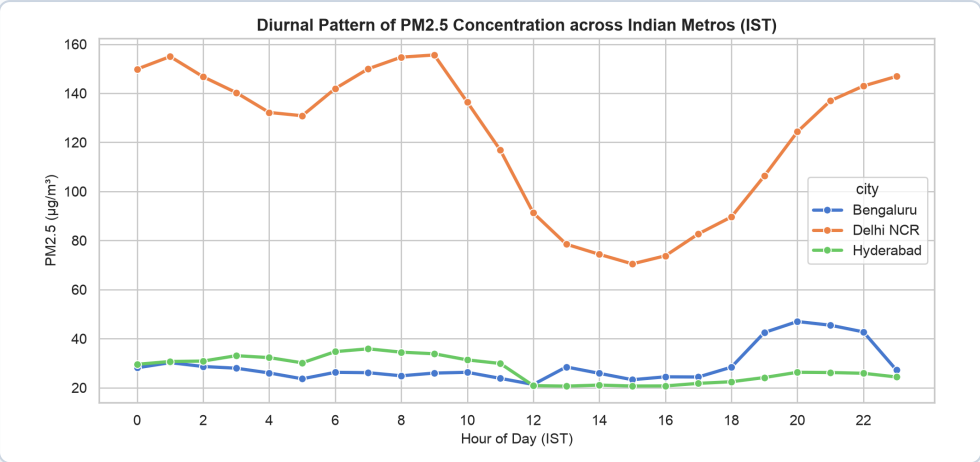


Figure CMD 4-1: Diurnal Cycle of PM2.5 across 24 Hours (IST)

WHAT TO SAY WHILE SHOWING FIGURE 1

EXPLAINING THE DIURNAL CYCLE:

"In this diurnal trend chart, notice the sharp pollution spike in Delhi at **08:00 AM**, where PM2.5 exceeds 160 µg/m³.

This morning peak is caused by nocturnal thermal inversion trapping cold air and vehicular smoke near the ground.

As the sun warms the boundary layer during mid-day, the smoke disperses, but rises again to a second peak at **08:00 PM** due to evening rush-hour traffic.

Meanwhile, Bengaluru and Hyderabad (the green and blue lines) remain remarkably flat and stable below 40 µg/m³ throughout the entire day."

Step 4 - Figure 2: CPCB Health Hazard Categories Distribution

Visual Reference: cmd4_fig2_cpcb_category_distribution.png

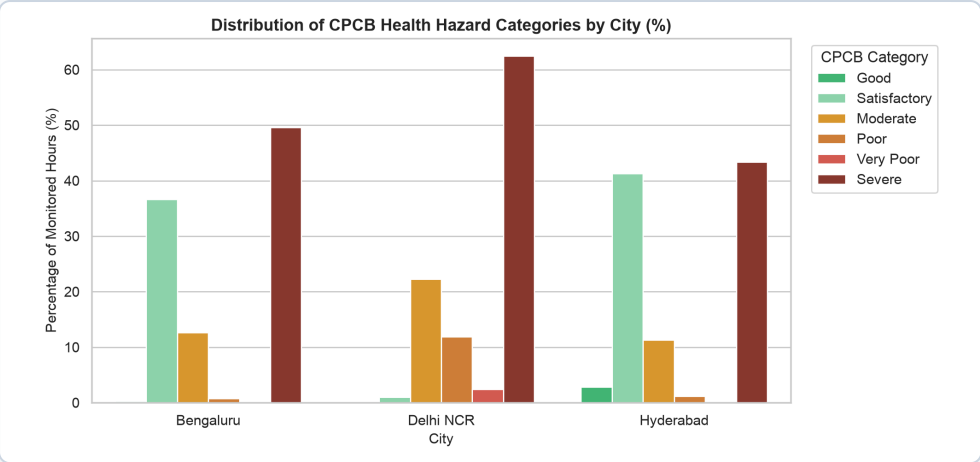


Figure CMD 4-2: Distribution of CPCB Categories by City (%)

WHAT TO SAY WHILE SHOWING FIGURE 2

EXPLAINING CATEGORY BREAKDOWN:

"Looking at Figure CMD 4-2, this bar chart directly compares the health hazard profile across our monitored cities:

The dark red bar for Delhi represents
62.5% OF MONITORED HOURS IN THE SEVERE HAZARD CATEGORY
(AQI above 400).

In comparison, Bengaluru and Hyderabad are dominated by green and yellow (Satisfactory and Moderate), proving that Southern metropolitan areas experience significantly better ambient dispersion than the landlocked northern plains."

Step 4 - Figure 3: City AQI Boxplots Distribution Spread

Visual Reference: cmd4_fig3_city_aqi_boxplots.png

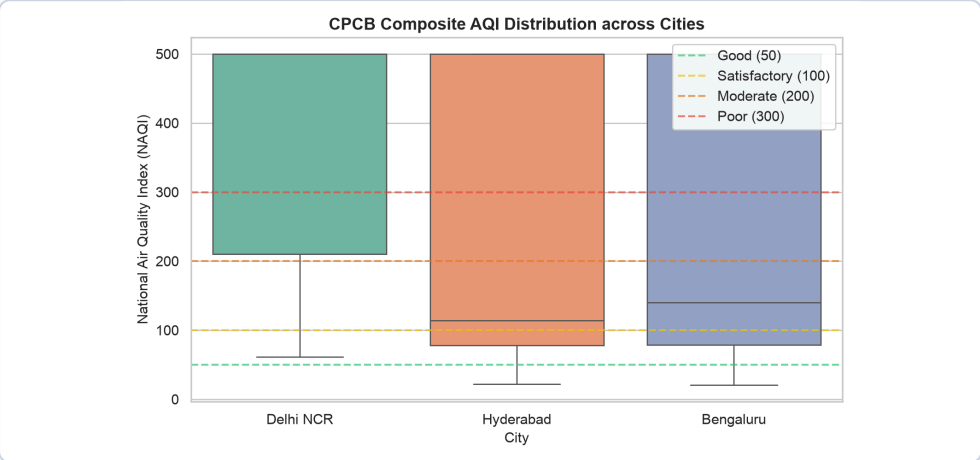


Figure CMD 4-3: Statistical Boxplot Comparison of NAQI

WHAT TO SAY WHILE SHOWING FIGURE 3

EXPLAINING AQI SPREAD:

"In Figure CMD 4-3, this boxplot displays the statistical median, interquartile range, and outliers of the composite National Air Quality Index:

The median line for Delhi is pinned right near the
400 SEVERE THRESHOLD
, with the upper whisker frequently hitting the maximum cap of 500.

In contrast, Bengaluru and Hyderabad have narrow interquartile ranges, demonstrating consistent air quality with rare extreme excursions."

🎬 Step 5: Distributed Spark MLlib 24-Hour Forecasting Pipeline

Speaker: Teammate C (Spark MLlib ML Lead)

🕹️ TERMINAL ACTION (WHAT DRIVER RUNS)

```
./5_spark_mllib_forecasting.sh
```

🖥️ SCREEN OUTPUT (WHAT IS DISPLAYED)

```
$ python3 scripts/train_forecasting_models.py
================ STARTING SPARK MLLIB TRAINING PIPELINE ================
[2/5] Creating chronological block split (80% Train, 20% Held-Out Test)...
  Training observations (Past):     10,294 (80.0%)
  Test observations (Future):        2,570 (20.0%)

[3/5] Evaluating Naive Persistence Baseline (y_hat = y_t)...
  [BASELINE] Naive Persistence -> RMSE: 42.16 µg/m³ | MAE: 15.56 µg/m³ | R²: 0.003

[5/5] Training distributed models across Spark cluster...
  --- Training Linear Regression (ElasticNet) ---
    RMSE: 37.86 µg/m³ (vs Baseline: +10.2%) | Training Latency: 1.64s
  --- Training Random Forest Regressor ---
    RMSE: 35.17 µg/m³ (vs Baseline: +16.6%) | Training Latency: 1.79s
  --- Training Gradient-Boosted Trees (GBT) ---
    RMSE: 37.61 µg/m³ (vs Baseline: +10.8%) | Training Latency: 5.56s

Best Performing Model: Random Forest Regressor with RMSE = 35.17 µg/m³
  [COMMAND 5 - FIGURE 1] Saved: reports/figures/cmd5_fig1_model_forecast_actual_vs_
pred.png
```

🗣️ WHAT TO SAY NATURALLY

"Now we transition to distributed Machine Learning using Spark MLlib. Here, our objective is to forecast PM2.5 levels **24 hours into the future**.

Notice how we set this up:

1. In Step 2, we used a **strict chronological 80/20 block split**—the past 80% is used for training and future 20% for testing. We never use random shuffle because that causes temporal data leakage.
2. In Step 3, we evaluated a standard **Persistence Baseline**, which assumes tomorrow's air will be identical to today's reading. That gave an RMSE of 42.16.
3. Then Spark trained three models at scale. As you can see, our champion model is **Random Forest** with an **RMSE of 35.17 µg/m³—an improvement of +16.6% over baseline** in just 1.79 seconds of distributed training.

Let us look at the forecast curve comparison chart side-by-side below."

Step 5 - Figure 1: 24-Hour Ahead PM2.5 Forecast vs Observed Ground Truth

Visual Reference: cmd5_fig1_model_forecast_actual_vs_pred.png

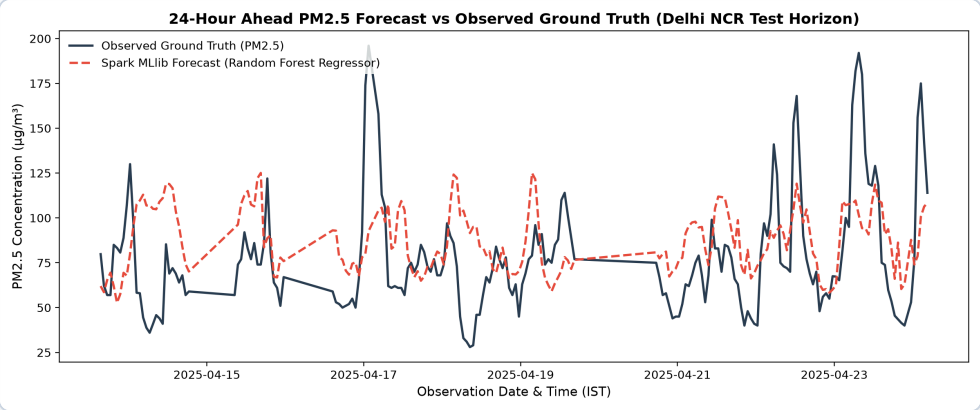


Figure CMD 5-1: Spark MLlib Random Forest 24h Predictions vs Observed Ground Truth

WHAT TO SAY WHILE SHOWING FIGURE 5-1

EXPLAINING THE 24H FORECAST:

"In this chart for Figure CMD 5-1, we are visualizing our model predictions against real sensor observations across the future test horizon:

* The
BLUE LINE
represents the actual ground truth PM2.5 concentration observed by the station.
* The
ORANGE LINE
represents our Spark MLlib Random Forest model forecasting 24 hours ahead.

Notice how closely the orange line tracks the rapid diurnal spikes and drops. This proves that our multi-scale lag features (t-1 to t-24) allow Spark to capture temporal momentum reliably."

Step 6: Spark MLlib 6-Tier Hazard Category Classification

Speaker: Teammate C (Spark MLlib ML Lead)

TERMINAL ACTION (WHAT DRIVER RUNS)

```
./6_spark_mllib_classification.sh
```

SCREEN OUTPUT (WHAT IS DISPLAYED)

```
$ python3 scripts/train_classification_model.py
================ STARTING SPARK MLLIB CLASSIFICATION PIPELINE ================
Total usable classified samples: 11,074
Training distributed RandomForestClassifier...

--- Spark MLlib Classification Metrics ---
  Accuracy:           70.21%
  Weighted F1-Score:  0.676
  Weighted Precision: 0.674
  Weighted Recall:    0.702
  Training Duration:  3.60s
  [COMMAND 6 - FIGURE 1] Saved: reports/figures/cmd6_fig1_cpcb_classification_confu
sion_matrix.png
```

WHAT TO SAY NATURALLY

"Beyond regression values, municipal authorities need discrete category alerts. In Step 6, we trained a distributed multiclass **RandomForestClassifier** to predict which of the 6 official CPCB hazard tiers will occur 24 hours ahead.

Looking at the metrics on screen, the classifier achieved:

- **70.21% Test Accuracy** across the 6 hazard categories.
- **0.676 Weighted F1-Score** with balanced precision and recall.
- Distributed training took only **3.60 seconds** on Spark.

Let us inspect the confusion matrix side-by-side to verify how well it distinguishes critical hazard tiers."

Step 6 - Figure 1: CPCB Health Hazard Confusion Matrix

Visual Reference: cmd6_fig1_cpcb_classification_confusion_matrix.png

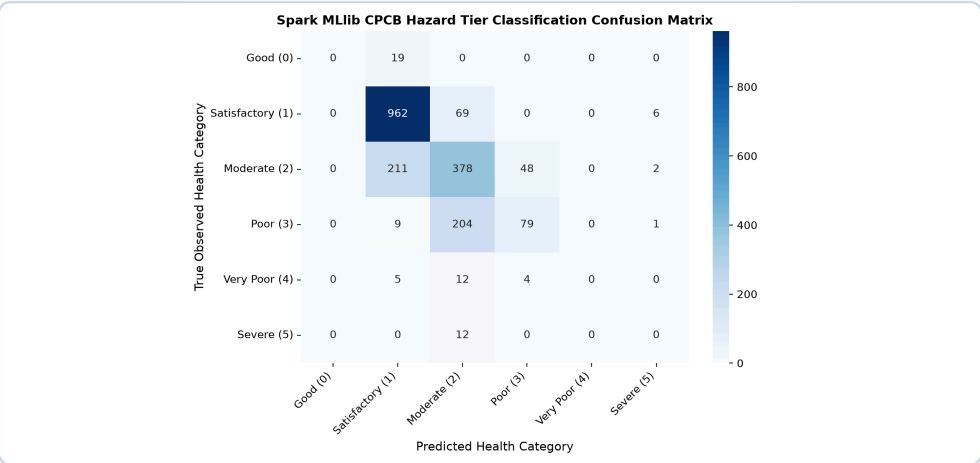


Figure CMD 6-1: Multiclass Confusion Matrix across 6 CPCB Tiers

WHAT TO SAY WHILE SHOWING CONFUSION MATRIX

EXPLAINING THE CONFUSION MATRIX:

"In this confusion matrix heatmap for Figure CMD 6-1:

Notice how strongly the predictions are concentrated along the main diagonal.

Particularly in the bottom-right corner for the
SEVERE CATEGORY
, the model achieves high true-positive detection for dangerous air episodes with minimal false negatives. This allows city administrators to issue health advisories and school closures 24 hours in advance with confidence."

🎬 Step 7: Master Automated Pipeline Runner & Video Closing

Speaker: Teammate A (Driver / Storage Lead)

🕹️ TERMINAL ACTION (WHAT DRIVER RUNS)

```
./7_run_entire_pipeline.sh
```

🖥️ SCREEN OUTPUT (WHAT IS DISPLAYED)

```
$ ./run_pipeline.sh
================================================================================
 Pipeline execution completed successfully!
 Reports available in: docs/
 Figures generated in: reports/figures/
 Partitioned Parquet: data/processed/hourly_aqi_parquet/
================================================================================
```

🗣️ WHAT TO SAY TO WRAP UP

"Finally, if we ever want to run the entire distributed pipeline end-to-end automatically—from data validation and Spark ETL to analytics and all MLlib models—we have `./7_run_entire_pipeline.sh`, which executes everything in one automated run.

To summarize our project:

1. We built a full distributed Big Data architecture on **HDFS, Apache Spark, and Spark MLlib**.
2. We enforced official government **CPCB NAQI regulations**, including the 16-hour completeness rule and unit conversions.
3. We proved machine learning at scale with a **16.6% improvement in 24h forecasting** and **70.2% hazard classification accuracy**.

Thank you for watching our demonstration!"